Task Management Application

Complete Full Stack Project Documentation

Introduction

The Task Management Application is a full-stack web application developed using MERN stack technologies. The main purpose of this project is to help users manage their daily tasks in a simple and efficient way. Users can register, login, create tasks, update tasks, and delete tasks.

Objective

The main objective of this project is to develop a task management system where users can manage tasks online. The application allows users to register account, login securely, add tasks, update tasks, delete tasks, and view all tasks.

Technologies Used

Frontend: React.js, HTML, CSS, Bootstrap

Backend: Node.js, Express.js

Database: MongoDB

Authentication: JWT Authentication

Additional Packages: Axios, Nodemon, Dotenv, Bcryptjs

Main Features

• User Registration
• User Login Authentication
• Create Tasks
• Update Tasks
• Delete Tasks
• Responsive User Interface
• MongoDB Database Integration
• JWT Security

Project Folder Structure

```
task-manager-app/

■■■ client/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■   ■■■ Navbar.js
■   ■   ■   ■■■ TaskForm.js
■   ■   ■   ■■■ TaskList.js
■   ■   ■■■ pages/
■   ■   ■   ■■■ Login.js
■   ■   ■   ■■■ Register.js
■   ■   ■   ■■■ Dashboard.js
■   ■   ■■■ App.js
■   ■   ■■■ api.js
■   ■   ■■■ index.js
■   ■■■ package.json
■
■■■ server/
■   ■■■ models/
■   ■   ■■■ User.js
■   ■   ■■■ Task.js
■   ■■■ routes/
■   ■   ■■■ authRoutes.js
■   ■   ■■■ taskRoutes.js
■   ■■■ middleware/
■   ■   ■■■ authMiddleware.js
■   ■■■ server.js
■   ■■■ package.json
```

Frontend Setup

```
npx create-react-app client
cd client
npm install axios react-router-dom bootstrap
```

Bootstrap Setup - index.js

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

API Configuration - api.js

```
import axios from "axios";

export default axios.create({
  baseURL: "http://localhost:5000/api"
});
```

Frontend Routing - App.js

```
import React from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";

import Login from "./pages/Login";
import Register from "./pages/Register";
import Dashboard from "./pages/Dashboard";
```

```
function App() {

  return (
    <BrowserRouter>

      <Routes>

        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/dashboard" element={<Dashboard />} />

      </Routes>

    </BrowserRouter>
  );
}

export default App;
```

Login Page - Login.js

```
import React, { useState } from "react";
import api from "../api";

function Login() {

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  const loginUser = async () => {

    const res = await api.post("/auth/login", {
      email,
      password
    });

    localStorage.setItem("token", res.data.token);

    alert("Login Successful");
  };

  return (
    <div>

      <h2>Login</h2>

      <input
        type="email"
        placeholder="Enter Email"
        onChange={(e) => setEmail(e.target.value)}
      />

      <input
        type="password"
        placeholder="Enter Password"
        onChange={(e) => setPassword(e.target.value)}
      />

      <button onClick={loginUser}>
        Login
      </button>

    </div>
  );
}
export default Login;
```

Dashboard Page - Dashboard.js

```
import React, { useEffect, useState } from "react";
import api from "../api";

function Dashboard() {

  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState("");

  useEffect(() => {
    fetchTasks();
  }, []);

  const fetchTasks = async () => {

    const res = await api.get("/tasks");

    setTasks(res.data);
  };

  const addTask = async () => {

    await api.post("/tasks", {
      title
    });

    fetchTasks();
  };

  return (
    <div>

      <h2>Task Dashboard</h2>

      <input
        type="text"
        placeholder="Enter Task Title"
        onChange={(e) => setTitle(e.target.value)}
      />

      <button onClick={addTask}>
        Add Task
      </button>

      {
        tasks.map((task) => (

          <div key={task._id}>
            <h4>{task.title}</h4>
          </div>

        ))
      }

    </div>
  );
}

export default Dashboard;
```

Backend Setup

```
npm init -y
npm install express mongoose cors dotenv bcryptjs jsonwebtoken nodemon
```

Backend Server - server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

require("dotenv").config();

const authRoutes = require("./routes/authRoutes");
const taskRoutes = require("./routes/taskRoutes");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)

.then(() => console.log("MongoDB Connected"))

.catch((err) => console.log(err));

app.use("/api/auth", authRoutes);
app.use("/api/tasks", taskRoutes);

const PORT = 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

User Model - User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({

  name: String,
  email: String,
  password: String

});

module.exports = mongoose.model("User", userSchema);
```

Task Model - Task.js

```
const mongoose = require("mongoose");

const taskSchema = new mongoose.Schema({

  title: String,
  description: String,

  status: {
    type: String,
    default: "Pending"
  }
});
```

```
module.exports = mongoose.model("Task", taskSchema);
```

Authentication Routes - authRoutes.js

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

const User = require("../models/User");

const router = express.Router();

router.post("/register", async (req, res) => {

  const { name, email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  const user = new User({
    name,
    email,
    password: hashedPassword
  });

  await user.save();

  res.json({
    message: "User Registered Successfully"
  });

});

router.post("/login", async (req, res) => {

  const { email, password } = req.body;

  const user = await User.findOne({ email });

  if (!user) {

    return res.json({
      message: "User Not Found"
    });

  }

  const isMatch = await bcrypt.compare(password, user.password);

  if (!isMatch) {

    return res.json({
      message: "Invalid Password"
    });

  }

  const token = jwt.sign(
    { id: user._id },
    "secretkey"
  );

  res.json({ token });

});

module.exports = router;
```

Task Routes - taskRoutes.js

```
const express = require("express");

const Task = require("../models/Task");

const router = express.Router();

router.get("/", async (req, res) => {

  const tasks = await Task.find();

  res.json(tasks);

});

router.post("/", async (req, res) => {

  const task = new Task(req.body);

  await task.save();

  res.json(task);

});

router.put("/:id", async (req, res) => {

  const updatedTask = await Task.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );

  res.json(updatedTask);

});

router.delete("/:id", async (req, res) => {

  await Task.findByIdAndDelete(req.params.id);

  res.json({
    message: "Task Deleted Successfully"
  });

});

module.exports = router;
```

Environment Variables - .env

```
MONGO_URI=your_mongodb_connection

JWT_SECRET=secretkey
```

Optional Real-Time Updates

```
npm install socket.io
```

Run Backend Server

```
cd server
npm install
npm run dev
```

Run Frontend Application

```
cd client
npm install
npm start
```

Expected Output

After running the project successfully, users can register, login, create tasks, update tasks, delete tasks, and manage task details using a responsive dashboard interface.

Conclusion

The Task Management Application is a useful MERN stack project developed using React.js, Node.js, Express.js, and MongoDB. This project helps in understanding frontend and backend integration, authentication systems, REST API development, CRUD operations, and database connectivity.